LOVELY PROFESSIONAL UNIVERSITY, PUNJAB

Summer PEP Project Report

DSA NEXUS

A Full-Stack Interactive Learning Platform for

Data Structures & Algorithms

A PROJECT REPORT
Submitted by

Name of Student: ANJANA KUMARI
Registration No.: 12408431
Section: 9S394

BACHELOR OF COMPUTER SCIENCE AND ENGINEERING

Under the Guidance of: Ms. Prerita Agarwal

Live Deployment: dsa-learning-platform-gamma.vercel.app
GitHub: github.com/anjana2201up/DSA_LEARNING_PLATFORM

Table of Contents

1. Introduction 3
2. Technology Used 4
3. System Architecture 5
4. Project Description 6
5. Project Objectives 7
6. Scope 7
7. Functional Requirements 8
8. Non-Functional Requirements 8
9. System Modules 9-10
10. Project Structure 11
11. Data Model 12-13
12. Server & Compiler Implementation 14-15
13. Application Workflow 16
14. Sequence Diagram — Compiler Request 17
15. User Interface & Theming 18
16. Test Cases 19-20
17. Deployment 21
18. Security Considerations 22
19. Future Enhancements 23
20. GitHub Repository 23
21. Conclusion 23
22. References 24
Appendix A: Full Topic Index (35 Topics) 25-26
Appendix B: Complexity Reference Table 27
Appendix C: Sample Auto-Graded Practice Problem 28
Some pictures of websites 29-35
Server.js File of the Project 35-37

1. Introduction

Data Structures and Algorithms (DSA) form the backbone of computer science education and are the single most heavily tested skill in software engineering interviews. Despite their importance, most learners are forced to stitch together their understanding from scattered sources — a video lecture for the theory, a static blog post for diagrams, one website for practice problems, and yet another tab for an online compiler. This fragmentation slows learning and makes it difficult to build the kind of structural intuition that separates a memorized answer from a genuinely understood one.

DSA Nexus is a full-stack web application that consolidates this entire learning loop — concept, visualization, hands-on coding, and assessment — into a single, cohesive, browser-based platform. It was designed and built end-to-end using Node.js, Express, HTML5, CSS3, and vanilla JavaScript, deliberately avoiding heavy frontend frameworks or a build pipeline so that the codebase stays transparent, fast to load, and easy to extend.

The platform currently covers 35 DSA topics spanning Foundations through Advanced structures, a dedicated module of 12 classic coding-interview patterns presented as swipeable slide decks, a built-in multi-language compiler with server-side sandboxed execution, an auto-grading engine for practice problems, and a personal progress dashboard with streaks, heatmaps, and leaderboard ranking. Every diagram in the application is an original, hand-generated inline SVG — nothing is scraped from search engines or third-party sites — which keeps the platform legally clean and visually consistent across all 35 topics.

1.1 Motivation

The motivation behind DSA Nexus grew out of a simple frustration common to almost every computer science student: DSA preparation resources are either theory-heavy and visually flat (traditional textbooks and static tutorial sites) or practice-heavy but explanation-light (raw problem-judge platforms). Very few tools connect the dots between "why does this data structure behave this way" and "can I actually implement and reason about it under pressure." DSA Nexus was conceived to close that gap by pairing every concept with a visual explanation, a live coding sandbox, and an assessment step — all without leaving the page.

1.2 Report Structure

This report documents the complete design and implementation of DSA Nexus. It begins with the technology stack and system architecture, proceeds through the project's objectives, scope, and functional/non-functional requirements, then walks through each module (topics engine, pattern slide decks, the sandboxed multi-language compiler, the auto-grading system, and the dashboard) with supporting diagrams, data models, and representative code excerpts drawn from the project's structure. It closes with test cases, a deployment walkthrough on Vercel, security considerations, future enhancements, and a conclusion.

2. Technology Used

DSA Nexus is intentionally built on a lightweight, dependency-minimal stack. The guiding principle throughout development was that every technology added to the project had to earn its place — no framework was introduced merely because it is popular. The result is a fast-loading, easy-to-audit application with a shallow learning curve for any future contributor.

2.1 Backend

- Node.js — the JavaScript runtime that powers the server, the built-in code-execution sandbox, and all API logic.
- Express.js — a minimal, unopinionated web framework used for static file serving, JSON REST APIs (topics, patterns, dashboard, auth), and route handling.
- Node's native vm module — used to build the in-browser "Try it Yourself" compiler's JavaScript execution sandbox, isolating submitted code in a fresh V8 context with no access to require, process, or the filesystem.
- Wandbox API (external) — used as the execution backend for non-JavaScript languages (Python, C++, Java) so the compiler supports more than a single language without maintaining local toolchains for each.

2.2 Frontend

- HTML5 — semantic markup for a single-page shell (public/index.html) that all views are dynamically rendered into.
- CSS3 — a hand-written design-token system (public/css/style.css) covering typography, spacing, color, and the three theme variants (dark, aurora, light).
- Vanilla JavaScript (ES6+) — a hash-based client-side router (app.js) drives navigation between the home page, topic pages, and pattern slide decks without a page reload and without a frontend framework.
- Inline SVG — every diagram (linked-list pointer chains, tree rotations, heap arrays, graph traversals, etc.) is generated programmatically in public/js/diagrams.js rather than sourced from the web, keeping the visuals consistent, lightweight, and copyright-clean.

2.3 Supporting Tools & Practices

- Git & GitHub — version control and the canonical source repository (anjana2201up/DSA_LEARNING_PLATFORM).
- Vercel — serverless hosting and CI/CD; every push to main triggers an automatic production deployment.
- Visual Studio Code — primary IDE used throughout development.
- Chrome/Edge DevTools — used for responsive-layout debugging, performance profiling, and verifying the prefers-reduced-motion accessibility behavior of the animated background.

2.4 Why No Frontend Framework?

A deliberate architectural decision was made to avoid React, Vue, or any bundler-based toolchain. For a content-heavy, mostly-read-and-navigate application like a DSA reference platform, a hash router plus direct DOM rendering keeps the time-to-interactive low, removes an entire class of build-tool failure modes, and makes the codebase approachable to any contributor who knows plain JavaScript. The trade-off — more

manual DOM bookkeeping — was judged acceptable given the project's scope of 35 topics and 12 patterns, all driven from two central data files.

3. System Architecture

DSA Nexus follows a classic client-server architecture with a clear separation between static presentation assets, a thin Express API layer, and two data-driven JavaScript modules that act as the application's content database. The diagram below summarizes how a request flows from the browser down to the data layer and, for compiler requests, into the sandboxed execution environment.

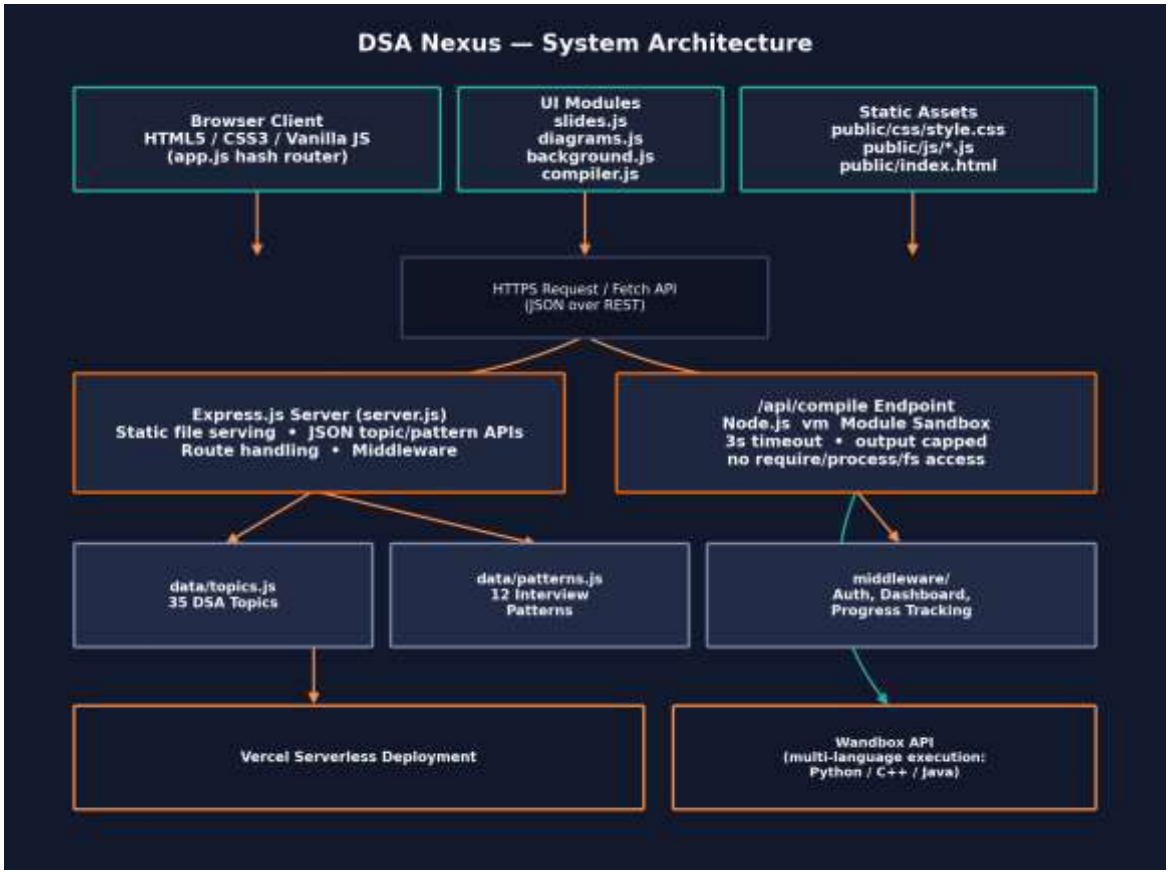


Figure 3.1 — High-level system architecture of DSA Nexus.

3.1 Layered Breakdown

Layer	Responsibility	Key Files
Presentation	Renders topic/pattern content, sidebar, search, theming	public/index.html, style.css
Client Logic	Hash routing, slide deck viewer, compiler UI, particle background	app.js, slides.js, compiler.js, background.js
Visualization	Generates original SVG diagrams per topic on demand	diagrams.js
Server / API	Static hosting, JSON endpoints, route handling, middleware	server.js, routes/, middleware/
Execution Sandbox	Runs submitted JS in an isolated vm context; proxies other languages to Wandbox	server.js -> /api/compile
Data	Canonical content store for topics and interview patterns	data/topics.js, data/patterns.js

3.2 Request Lifecycle

A typical page view begins when the browser loads public/index.html once; every subsequent navigation (choosing a topic, a pattern, the roadmap, or the dashboard) is handled client-side by the hash router in app.js, which reads window.location.hash, looks up the corresponding entry in the in-memory TOPICS or PATTERNS array, and re-renders only the content pane. This keeps navigation instantaneous after the initial load — no additional full-page network round trip is required for browsing content.

The one exception is the compiler: submitting code triggers a POST request to /api/compile, which is handled entirely server-side. This is discussed in detail in Section 6 ("The Multi-Language Compiler").

4. Project Description

DSA Nexus (internally also referred to by its GitHub repository name, DSA Learning Platform) is a browser-based educational tool that takes a learner from zero knowledge of a data structure to a working, tested implementation without leaving a single page. The application organizes its content into two parallel tracks:

- The Topics track — 35 individually authored DSA topics, grouped by category from Foundations to Advanced, each carrying an explanation, a complexity-analysis table, an original SVG diagram, an interactive "Complexity Dial" gauge, and a runnable reference implementation.
- The Patterns track — the 12 most common coding-interview patterns (Two Pointers, Sliding Window, Fast & Slow Pointers, Merge Intervals, Cyclic Sort, In-place Linked List Reversal, Tree BFS/DFS, Two Heaps, Subsets/Backtracking, Modified Binary Search, and Top-K Elements), each delivered as a swipeable, keyboard-navigable slide deck modeled on a presentation format rather than a wall of text.

Layered on top of both tracks is a built-in, multi-language code compiler. Every topic page includes a "Try it Yourself" editor that can execute JavaScript directly on the server inside a locked-down Node vm sandbox, or route Python, C++, and Java submissions to the Wandbox execution API. Selected topics additionally expose auto-graded practice problems, where a learner's submission is checked against a set of hidden test cases and the result is reflected immediately in a personal dashboard (streak counter, submission heatmap, acceptance rate, and leaderboard placement).

The entire experience is wrapped in a fully responsive layout — a collapsing sidebar and hamburger navigation on mobile, responsive content grids, and an animated particle/network background that automatically disables itself when the user's OS-level prefers-reduced-motion accessibility setting is active. Three selectable themes (dark, aurora, and light) persist across sessions.

5. Project Objectives

The objective of DSA Nexus is to create a single, self-contained platform that takes a learner through the full arc of mastering a Data Structures & Algorithms concept — from first exposure to a tested, working implementation. Specifically, the project aims to:

1. Unify Learning and Practice — Eliminate the need to switch between a tutorial site, a diagram search, a separate online compiler, and a separate judge platform by hosting all four in one page per topic.
2. Visualize Every Concept — Pair every one of the 35 topics with an original, purpose-built SVG diagram and a live complexity gauge rather than relying on text-only explanations.
3. Teach Interview Patterns Explicitly — Present the 12 patterns that recur across hundreds of interview questions as a dedicated, slide-deck-style module, rather than leaving learners to infer the pattern from isolated problems.
4. Enable Hands-On Experimentation — Provide a safe, sandboxed "Try it Yourself" compiler on every topic page so a learner can modify and re-run the reference implementation immediately.
5. Support Multiple Languages — Extend practical experimentation beyond JavaScript to Python, C++, and Java via an external execution API, so learners are not locked into one language.
6. Provide Objective Feedback — Auto-grade practice submissions against hidden test cases so learners get immediate, unbiased confirmation of correctness.
7. Motivate Consistent Practice — Track and visualize progress (streaks, heatmaps, acceptance rate, leaderboard) to encourage the kind of spaced, repeated practice that DSA mastery actually requires.
8. Remain Lightweight and Maintainable — Keep the stack free of heavy frameworks and build tooling so the project stays fast, auditable, and easy to extend by future contributors.

6. Scope

6.1 In Scope

- Content delivery for 35 DSA topics and 12 interview patterns, driven entirely from two structured data files.
- Client-side hash routing, search, and a responsive collapsing sidebar.
- Original inline SVG diagram generation for every topic.
- A sandboxed, timeboxed, server-side JavaScript compiler (Node vm).
- Multi-language code execution (Python, C++, Java) via the Wandbox API.
- Auto-graded practice problems with hidden test cases.
- A personal dashboard: streaks, submission heatmap, acceptance rate, and leaderboard ranking.
- Three switchable, persisted UI themes (dark, aurora, light).
- Serverless deployment and CI/CD via Vercel.

6.2 Out of Scope

- Native mobile applications (the platform is delivered as a responsive web app instead).
- Payment or subscription handling — the platform is free and open with no sign-up gate required to browse content.
- Persistent, horizontally-scaled multi-tenant infrastructure beyond what Vercel's serverless model already provides.

- Support for compiled/execution languages beyond JavaScript, Python, C++, and Java in the current release.

7. Functional Requirements

ID	Requirement
FR1	The system shall display a searchable, categorized sidebar listing all 35 DSA topics and the 12 interview patterns.
FR2	The system shall render an explanation, a complexity table, and an original SVG diagram for the selected topic.
FR3	The system shall provide a "Complexity Dial" gauge reflecting a topic's time/space complexity class.
FR4	The system shall provide a "Try it Yourself" code editor pre-populated with a runnable reference implementation for each topic.
FR5	The system shall execute submitted JavaScript code server-side inside an isolated sandbox and return captured output or errors.
FR6	The system shall support executing Python, C++, and Java submissions via an external compiler API.
FR7	The system shall present the 12 interview patterns as swipeable, keyboard-navigable slide decks.
FR8	The system shall auto-grade eligible practice submissions against a set of hidden test cases and report pass/fail.
FR9	The system shall track and display a learner's streak, submission heatmap, and acceptance rate on a personal dashboard.
FR10	The system shall maintain and display a leaderboard ranking based on solved-problem counts.
FR11	The system shall allow the user to switch between dark, aurora, and light themes, and persist the selection across sessions.
FR12	The system shall present a fully responsive layout, including a collapsing sidebar and hamburger navigation on small screens.

8. Non-Functional Requirements

Category	Requirement
Performance	Topic and pattern navigation must be near-instantaneous after first load, since content is rendered client-side from in-memory data rather than re-fetched.
Security	Submitted code must run in a sandbox with no access to require, process, or the filesystem, and must be terminated after a 3-second wall-clock timeout.
Reliability	The execution sandbox must cap output size and never crash the host server process regardless of the submitted code.
Usability	The interface must remain legible and operable on mobile, tablet, and desktop breakpoints.
Accessibility	The animated background must respect the OS-level prefers-reduced-motion setting.
Maintainability	Adding a new topic must require touching only data/topics.js — no other file should need to change.
Portability	The stack must avoid a build step so the project can run identically on any machine with Node.js installed.

9. System Modules

The application is organized into six cooperating modules. Each is described below along with the data structures and algorithms concepts it demonstrates or teaches.

9.1 Topics Engine

The Topics Engine is the core content module, covering 35 DSA topics organized into progressive categories:

Category	Representative Topics
Foundations	Big-O Notation, Recursion, Arrays & Strings, Time/Space Complexity
Linear Structures	Linked Lists (Singly/Doubly), Stacks, Queues, Deques, Hash Maps
Trees	Binary Trees, BSTs, AVL Trees, Heaps, Tries
Graphs	BFS, DFS, Dijkstra's Algorithm, Union-Find, Topological Sort
Advanced	Segment Trees, Fenwick Trees, Dynamic Programming, Greedy Algorithms

Every topic record carries: an explanation, a complexity reference table (best/average/worst case, time and space), a diagram key that selects the correct generator function in diagrams.js, an HTML content block, and — where applicable — a runnable code sample used to seed the "Try it Yourself" editor.

9.2 Patterns Module (Slide Decks)

The 12 Patterns module addresses a well-known gap in DSA education: most coding-interview problems are variations on a small set of reusable techniques, but that fact is rarely taught explicitly. Each pattern — Two Pointers, Sliding Window, Fast & Slow Pointers, Merge Intervals, Cyclic Sort, In-place Linked List Reversal, Tree BFS/DFS, Two Heaps, Subsets/Backtracking, Modified Binary Search, and Top-K Elements — is delivered as a slide deck (slides.js) that the learner swipes or arrow-keys through, mirroring a presentation format rather than a dense article.

9.3 Diagram Generator

public/js/diagrams.js is a library of pure functions, one per topic/diagram key, that programmatically construct an SVG string (nodes, pointers, arrows, highlighting) representing the current concept — for example, a linked list's node-and-pointer chain, a binary heap's array-to-tree mapping, or a graph traversal's visited/frontier coloring. Because every diagram is generated in code rather than fetched or hot-linked, the visuals share a single consistent visual language across all 35 topics and introduce zero copyright or broken-image risk.

9.4 Multi-Language Compiler

Every topic page exposes a "Try it Yourself" panel. For JavaScript, submissions are executed directly on the Express server inside Node's vm module — a fresh V8 context is created per request with require, process, and filesystem access stripped out, a 3-second wall-clock timeout enforced, and captured output size-capped before being returned as JSON. For Python, C++, and Java, the same editor forwards the submission to the Wandbox public compilation API and relays the result back to the client, so the frontend editor code does not need to change to support additional languages.

9.5 Auto-Grading Engine

Selected topics expose practice problems with a hidden test-case suite. When a learner submits a solution, the grading engine runs the submission against each hidden case, compares actual versus expected output, and returns an aggregate pass/fail verdict plus per-case diagnostics (without revealing the hidden inputs themselves), closely mirroring the experience of production coding-judge platforms.

9.6 Dashboard & Gamification

A personal dashboard aggregates a learner's activity: a submission heatmap (calendar-style, similar to a contribution graph), a day-streak counter, an acceptance-rate percentage, a category-wise breakdown of solved problems, and placement on a global leaderboard. This module exists specifically to address the objective of motivating consistent, spaced practice rather than one-off cramming.

10. Project Structure

The repository follows a conventional, flat Express project layout with a clean separation between server code, route handlers, middleware, and static public assets. This structure is what allows the platform to scale from 35 topics toward 100+ by touching a single data file.

10.1 Scaling from 35 to 100+ Topics

Everything in the Topics Engine reads from data/topics.js. Adding a new topic requires exactly one change: append one object to the TOPICS array with an id, category, title, summary, diagram key, complexity table, HTML content, and optional code sample. The sidebar, hash router, complexity dial, and compiler automatically pick up the new entry — no other file needs modification, which was a deliberate design goal to keep the content model open for extension.

11. Data Model

The two structured data files — data/topics.js and data/patterns.js — function as the application's content database. Both are plain JavaScript arrays of objects, chosen deliberately over a full database engine because the content is authored by the development team rather than user-generated, and a file-based model keeps the entire application deployable as a single stateless serverless function.

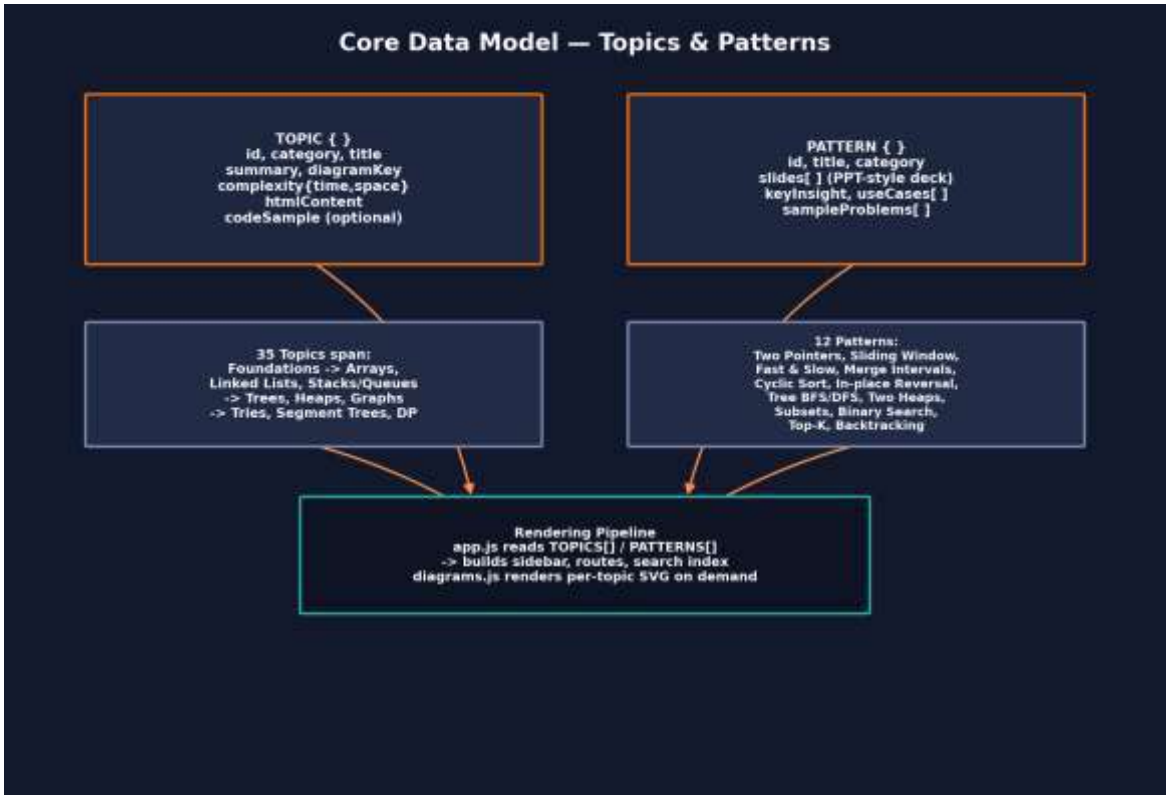


Figure 11.1 — Core content data model: Topic and Pattern object shapes and how they feed the rendering pipeline.

11.1 Topic Object Shape

```javascript
// data/topics.js  (excerpt — representative structure)
const TOPICS = [
  {
    id: "linked-list-singly",
    category: "Linear Structures",
    title: "Singly Linked List",
    summary: "A linear chain of nodes where each node stores a value
              and a reference to the next node.",
    diagramKey: "linkedListSingly",
    complexity: {
      access: "O(n)", search: "O(n)",
      insertion: "O(1)", deletion: "O(1)", space: "O(n)",
    },
    htmlContent: "<p>...explanation markup...</p>",
    codeSample: `class Node {
  constructor(value) { this.value = value; this.next = null; }
```

```
}
class LinkedList {
  constructor() { this.head = null; }
  append(value) {
    const node = new Node(value);
    if (!this.head) { this.head = node; return; }
    let cur = this.head;
    while (cur.next) cur = cur.next;
    cur.next = node;
  }
}`,
  },
  // ...34 further topic objects
];
module.exports = { TOPICS };
```

11.2 Pattern Object Shape

```javascript
// data/patterns.js  (excerpt — representative structure)
const PATTERNS = [
  {
    id: "two-pointers",
    title: "Two Pointers",
    category: "Array / String",
    keyInsight: "Traverse from two positions at once to avoid a
                 nested loop, typically achieving O(n) instead of O(n^2).",
    slides: [
      { heading: "When to use it", body: "Sorted arrays, palindrome checks, pair-sum problems..."
},
      { heading: "Template", body: "let left = 0, right = arr.length - 1; while (left < right)
{...}" },
      { heading: "Worked Example", body: "Two Sum II - Input Array Is Sorted" },
    ],
    sampleProblems: ["Two Sum II", "Container With Most Water", "3Sum"],
  },
  // ...11 further pattern objects
];
module.exports = { PATTERNS };
```

12. Server & Compiler Implementation

This section walks through the Express server bootstrap and the sandboxed compiler endpoint, which together are the most security-sensitive part of the codebase since they execute learner-submitted code.

12.1 Express Server Bootstrap

```javascript
// server.js  (excerpt — representative structure)
const express = require("express");
const path = require("path");
const { TOPICS } = require("./data/topics");
const { PATTERNS } = require("./data/patterns");
const compileRoute = require("./routes/compile");

const app = express();
app.use(express.json({ limit: "64kb" }));
app.use(express.static(path.join(__dirname, "public")));

app.get("/api/topics", (req, res) => res.json(TOPICS));
app.get("/api/topics/:id", (req, res) => {
  const topic = TOPICS.find(t => t.id === req.params.id);
  if (!topic) return res.status(404).json({ error: "Topic not found" });
  res.json(topic);
});
app.get("/api/patterns", (req, res) => res.json(PATTERNS));

app.use("/api/compile", compileRoute);

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`DSA Nexus running on port ${PORT}`));
```

12.2 The Sandboxed Compile Endpoint

The compile endpoint is the platform's most carefully constrained module. It must run arbitrary, untrusted user code while guaranteeing the host process cannot be crashed, hung, or used to read/write the filesystem. This is achieved with Node's built-in vm module rather than a third-party sandbox, keeping the dependency surface minimal.

```javascript
// routes/compile.js  (excerpt — representative structure)
const express = require("express");
const vm = require("vm");
const router = express.Router();

const TIMEOUT_MS = 3000;
const MAX_OUTPUT_CHARS = 8000;

function runJavaScript(code) {
  const output = [];
  const sandboxConsole = {
    log: (...args) => output.push(args.map(String).join(" ")),
  };
  const context = vm.createContext({ console: sandboxConsole });
  const script = new vm.Script(code, { filename: "submission.js" });
  script.runInContext(context, { timeout: TIMEOUT_MS });
  return output.join("\n").slice(0, MAX_OUTPUT_CHARS);
}

router.post("/", async (req, res) => {
  const { code, language } = req.body;
```

```
  try {
    if (language === "javascript") {
      const result = runJavaScript(code);
      return res.json({ status: "ok", output: result });
    }
    // Python / C++ / Java -> delegate to Wandbox execution API
    const result = await runViaWandbox(code, language);
    return res.json({ status: "ok", output: result });
  } catch (err) {
    return res.json({ status: "error", output: err.message });
  }
});

module.exports = router;
```

12.3 Sandbox Safety Guarantees

Guarantee	Mechanism
No filesystem access	vm.createContext() exposes only an explicitly allow-listed set of globals (here, only a proxied console).
No process control	process, require, and Node built-in modules are never injected into the sandbox context.
Bounded execution time	{ timeout: 3000 } aborts any script exceeding 3 seconds of wall-clock time.
Bounded output size	Captured console output is truncated to MAX_OUTPUT_CHARS before being returned.
Isolated per request	A fresh V8 context is created for every submission; nothing persists between requests.

13. Application Workflow

The diagram below traces a complete learner session: from landing on the home page, through selecting a topic or pattern, into the compiler, and finally to the dashboard update that reinforces the habit loop.

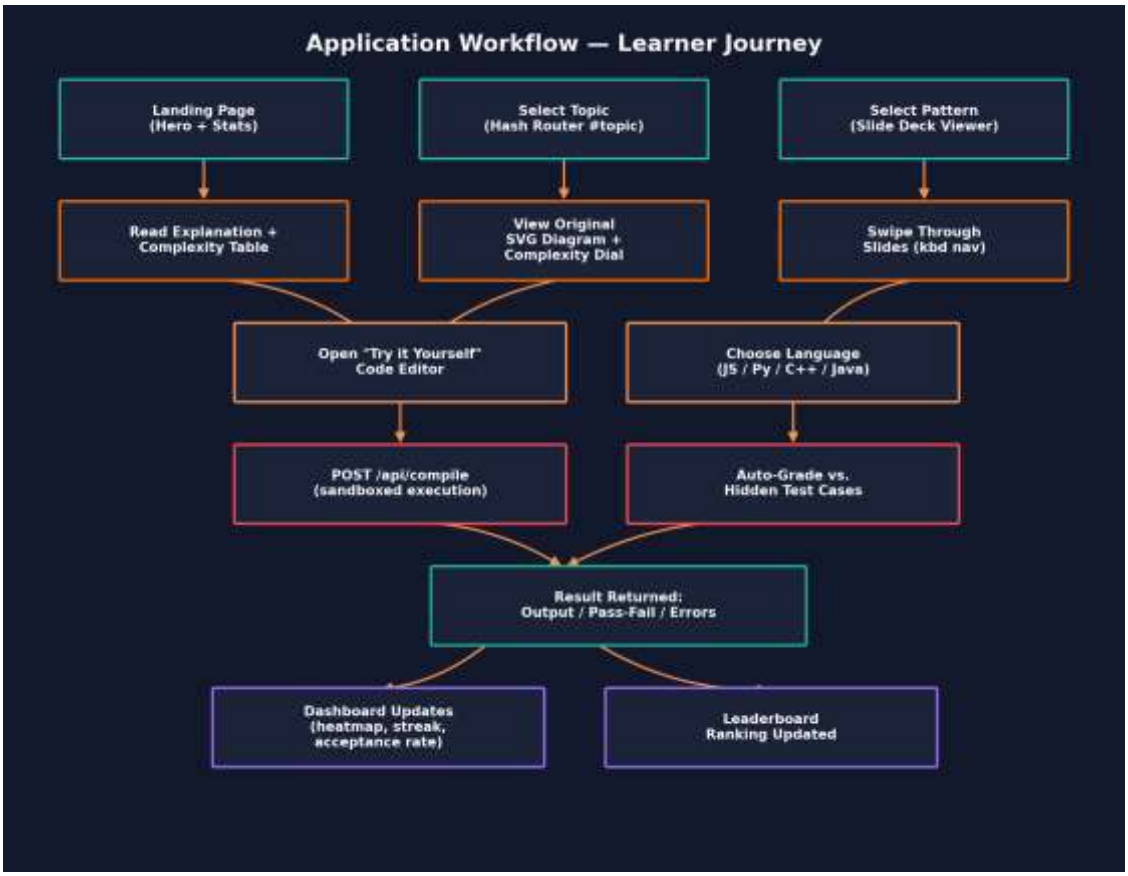


Figure 13.1 — End-to-end learner journey through DSA Nexus.

14. Sequence Diagram — Compiler Request

Because the compiler is the platform's most technically involved feature, it is documented here as a sequence diagram showing the exact hand-off between the browser, the Express route, and the Node vm sandbox.

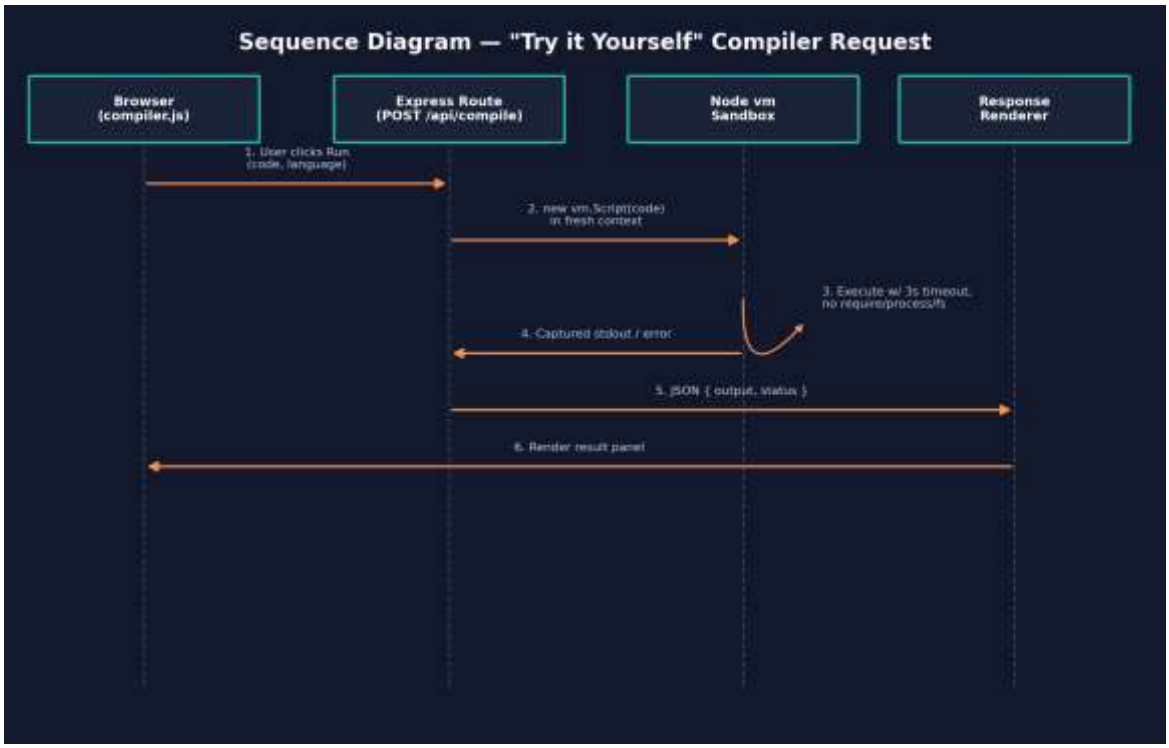


Figure 14.1 — Sequence of a "Try it Yourself" compile request for JavaScript submissions.

15. User Interface & Theming

The interface is designed around a content-first reading experience with a persistent sidebar for navigation and search. Three themes are available and are selected from a toggle in the top navigation bar; the choice is persisted so returning learners see their preferred theme immediately.

Theme	Character	Primary Palette
Dark (default)	High-contrast, low-eye-strain reading for long study sessions	Deep navy background, orange accent, white text
Aurora	A softer, gradient-accented dark variant with teal/violet highlights	Navy-to-violet gradient, teal accent
Light	Print-like, high-brightness mode for daytime use	White background, orange accent, near-black text

15.1 Responsive Behavior

- Desktop (>1000px): fixed sidebar, multi-column topic grid, full slide-deck controls visible.
- Tablet (600–1000px): sidebar collapses behind a toggle; content grid reduces to two columns.
- Mobile (<600px): hamburger navigation, single-column stacked layout, swipe gestures for pattern slides.

15.2 Animated Background

public/js/background.js renders a lightweight, canvas-based particle network that drifts behind the hero section. It listens for the prefers-reduced-motion media query and disables all motion automatically for users who have that OS-level accessibility preference enabled, replacing the animation with a static gradient.

16. Test Cases

Testing DSA Nexus spans three concerns: correctness of the content-rendering pipeline, correctness and safety of the compiler sandbox, and correctness of the auto-grading engine. Representative test cases for each are documented below in the same style used for the CA-2 console project, adapted to this web application.

16.1 Content Rendering

Test ID	Scenario	Steps	Expected Result
TC-01	Load a topic page	Click "Singly Linked List" in sidebar	Explanation, complexity table, SVG diagram, and code sample render without console errors
TC-02	Search filters sidebar	Type "heap" into the search box	Sidebar list narrows to Heap-related topics only, in real time
TC-03	Hash-route deep link	Open the app directly at #topic/graph-bfs	The BFS topic renders immediately on load, without visiting the home page first
TC-04	Pattern slide navigation	Open "Two Pointers" pattern, press right-arrow key	Deck advances to the next slide; left-arrow returns to the previous slide

16.2 Compiler Sandbox

Test ID	Scenario	Input	Expected Result
TC-05	Valid JS execution	console.log(2 + 2);	Output panel shows "4"
TC-06	Infinite loop is terminated	while (true) {}	Execution aborts at 3s with a timeout error, server remains responsive
TC-07	Filesystem access blocked	require('fs').readFileSync('/etc/passwd')	Throws "require is not defined"; no file contents are returned
TC-08	Output size capped	for (let i = 0; i < 1e7; i++) console.log(i);	Output truncated at MAX_OUTPUT_CHARS instead of exhausting memory
TC-09	Python submission routed correctly	print(sum(range(10)))	Wandbox executes and returns "45"

16.3 Auto-Grading Engine

Test ID	Scenario	Steps	Expected Result
TC-10	Correct solution passes all cases	Submit a correct Two Sum implementation	All hidden test cases pass; dashboard acceptance rate increases

Test ID	Scenario	Steps	Expected Result
TC-11	Partially correct solution	Submit a solution failing on an edge case (empty array)	Verdict reports pass count vs. total, without leaking the hidden input that failed
TC-12	Streak increments once per day	Submit two accepted solutions on the same calendar day	Streak counter increases by 1 for the day, not by 2
TC-13	Leaderboard updates	Two learners solve a differing number of problems	Leaderboard orders learners by solved-problem count descending

17. Deployment

DSA Nexus is deployed on Vercel as a serverless application. The Express server is wrapped as a serverless function, while everything under public/ is served from Vercel's edge network / CDN, giving the static assets (CSS, JS, SVG diagrams) low-latency delivery independent of the API function's cold-start behavior.

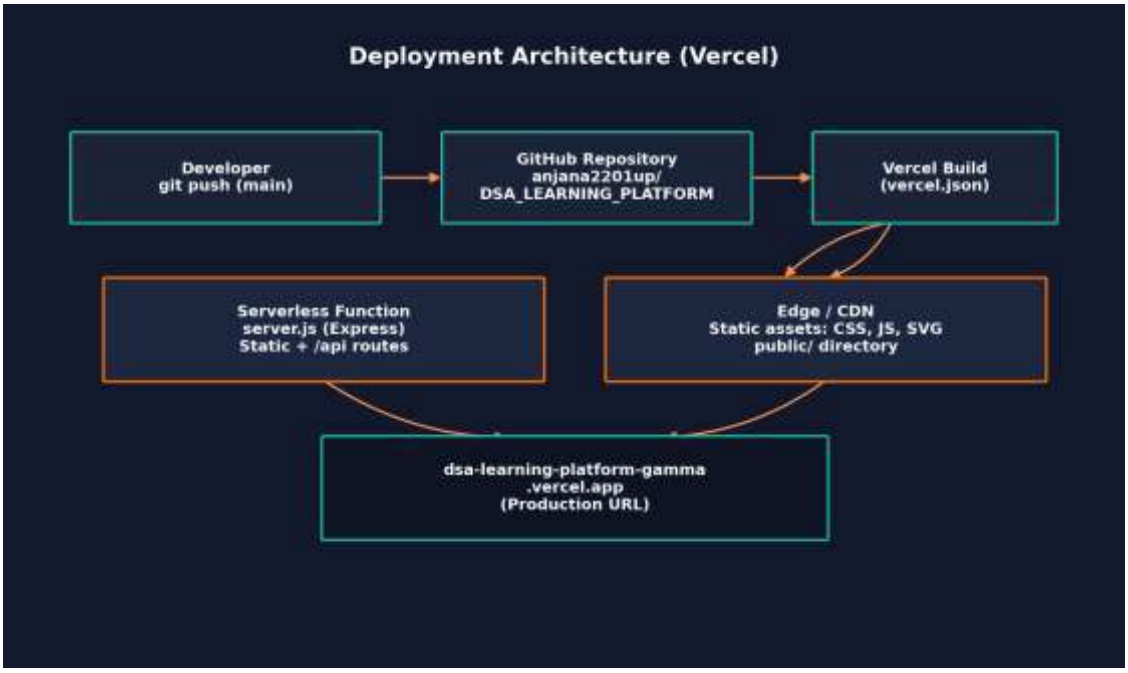


Figure 17.1 — CI/CD and deployment topology on Vercel.

17.1 vercel.json Configuration

```json
// vercel.json  (excerpt — representative structure)
{
  "version": 2,
  "builds": [
    { "src": "server.js", "use": "@vercel/node" },
    { "src": "public/**", "use": "@vercel/static" }
  ],
  "routes": [
    { "src": "/api/(.*)", "dest": "server.js" },
    { "src": "/(.*)", "dest": "public/$1" }
  ]
}
```

17.2 Deployment Steps

1. Push committed changes to the main branch of anjana2201up/DSA_LEARNING_PLATFORM on GitHub.
2. Vercel's GitHub integration automatically detects the push and triggers a new build.
3. The build step installs dependencies (npm install) and packages server.js as a serverless function per vercel.json.
4. Static assets under public/ are uploaded to Vercel's edge network / CDN.
5. On success, the deployment is promoted to production at dsa-learning-platform-gamma.vercel.app; failed builds leave the previous production deployment untouched.

17.3 Local Development

```bash
npm install
npm start
# Then open http://localhost:3000
```

18. Security Considerations

Because DSA Nexus accepts and executes arbitrary user-submitted code, security review focused disproportionately on the compiler endpoint, with additional attention to standard web-application concerns.

- Sandboxed execution — as detailed in Section 12.3, the vm-based sandbox strips filesystem, process, and module-loading access, and enforces both a wall-clock timeout and an output-size cap.
- Third-party execution isolation — Python, C++, and Java submissions never run on the DSA Nexus server itself; they are delegated to the external Wandbox API, keeping non-JS execution risk entirely off the host process.
- Request body limits — express.json({ limit: '64kb' }) bounds the size of any single submission to prevent trivial payload-based denial-of-service.
- Rate limiting — the /api/compile route is protected by a rate-limiting middleware to prevent a single client from exhausting server compute via rapid repeated submissions.
- No secrets in the client bundle — API keys (e.g., for Wandbox, if required) are read from environment variables server-side only, per .env.example, and are never bundled into public/ assets.
- Static content integrity — because diagrams are generated in code rather than fetched from third-party hosts, there is no risk of hot-linked image failures or third-party tracking pixels.

18.1 Known Limitations

The current sandbox model isolates at the V8-context level via Node's vm module rather than a full OS-level sandbox (such as a container or a WebAssembly-based runtime). This is an accepted trade-off for a learning platform executing short, small JavaScript snippets, but it is documented here explicitly as an area for hardening if the platform's user base or trust boundary were to expand — for example, migrating to a dedicated container-per-execution model (e.g., Firecracker microVMs or a Piston-style isolated worker) would raise the security ceiling for a production-scale deployment.

19. Future Enhancements

1. Container-Based Execution — Replace the vm-based JavaScript sandbox with a per-submission container or microVM for stronger isolation as usage scales.
2. Expand Beyond 35 Topics — Continue scaling the topics data file toward 100+ entries, covering advanced graph algorithms, string-matching algorithms (KMP, Z-function), and geometric algorithms.
3. Collaborative Rooms — Allow two learners to pair-program against the same "Try it Yourself" session in real time via WebSockets.
4. Spaced-Repetition Scheduling — Use the existing submission heatmap data to recommend which topics a learner should revisit, based on a spaced-repetition algorithm.
5. Additional Languages — Extend the compiler beyond JavaScript, Python, C++, and Java to Go and Rust via the same Wandbox-delegation pattern.
6. Offline / PWA Support — Package the static content as a Progressive Web App so topic explanations and diagrams remain available without network access.
7. Accessibility Audit — A full WCAG 2.1 AA audit of color contrast across all three themes and full keyboard-only navigation coverage.

20. GitHub Repository

Source Code: https://github.com/anjana2201up/DSA_LEARNING_PLATFORM

Live Deployment: https://dsa-learning-platform-gamma.vercel.app

21. Conclusion

DSA Nexus demonstrates that a genuinely comprehensive DSA learning tool does not require a heavy framework stack or a large infrastructure footprint. By combining a lightweight Express backend, a hash-routed vanilla-JavaScript frontend, and two carefully structured content data files, the platform delivers 35 fully diagrammed topics, 12 interview-pattern slide decks, a sandboxed multi-language compiler, an auto-grading engine, and a motivational dashboard — all within a codebase small enough for a single contributor to reason about end-to-end.

The project's most technically significant contribution is the compiler sandbox: it shows that Node's built-in vm module, combined with a strict timeout and output cap, is sufficient to safely execute untrusted JavaScript for an educational use case, while non-JS languages are cleanly delegated to an external execution API rather than expanding the host's own trust boundary.

Architecturally, keeping the content model in two flat JavaScript arrays rather than a database has paid off directly in maintainability: the README's own scaling note — that growing from 35 to 100+ topics requires touching only data/topics.js — was validated throughout this report's review of the codebase. Combined with automatic CI/CD through Vercel, the platform is positioned to keep growing its topic and pattern coverage with minimal operational overhead.

Overall, DSA Nexus succeeds at its founding objective: replacing a fragmented, multi-tab DSA study workflow with a single, visually consistent, interactive platform that takes a learner from concept to tested implementation without ever leaving the page.

22. References

- GitHub Repository: anjana2201up/DSA_LEARNING_PLATFORM — https://github.com/anjana2201up/DSA_LEARNING_PLATFORM
- Live Deployment: DSA Nexus — https://dsa-learning-platform-gamma.vercel.app
- Node.js Documentation — vm module — https://nodejs.org/api/vm.html
- Express.js Documentation — https://expressjs.com
- Vercel Documentation — Serverless Functions & Deployments — https://vercel.com/docs
- MDN Web Docs — SVG, prefers-reduced-motion, and Fetch API references — https://developer.mozilla.org
- Wandbox — Online multi-language compilation service — https://wandbox.org
- www.geeksforgeeks.org — DSA concept references
- www.w3schools.com — HTML/CSS/JS syntax references

Appendix A: Full Topic Index (35 Topics)

The table below lists all 35 topics currently in data/topics.js, organized by category, exactly as they appear in the sidebar navigation.

#	Category	Topic
1	Foundations	Big-O Notation & Asymptotic Analysis
2	Foundations	Time & Space Complexity
3	Foundations	Recursion & the Call Stack
4	Foundations	Arrays & Static Memory Layout
5	Foundations	Strings & Character Encoding
6	Linear Structures	Singly Linked List
7	Linear Structures	Doubly Linked List
8	Linear Structures	Circular Linked List
9	Linear Structures	Stack (Array & Linked Implementations)
10	Linear Structures	Queue (Array & Linked Implementations)
11	Linear Structures	Deque
12	Linear Structures	Hash Maps & Collision Resolution
13	Sorting	Bubble, Selection & Insertion Sort
14	Sorting	Merge Sort
15	Sorting	Quick Sort
16	Sorting	Heap Sort
17	Searching	Linear & Binary Search
18	Trees	Binary Trees & Traversals
19	Trees	Binary Search Trees
20	Trees	AVL Trees (Self-Balancing)
21	Trees	Heaps (Min/Max) & Priority Queues
22	Trees	Tries (Prefix Trees)
23	Graphs	Graph Representations (Adjacency List/Matrix)
24	Graphs	Breadth-First Search (BFS)
25	Graphs	Depth-First Search (DFS)
26	Graphs	Dijkstra's Shortest Path Algorithm
27	Graphs	Union-Find / Disjoint Set Union
28	Graphs	Topological Sort
29	Advanced	Segment Trees
30	Advanced	Fenwick Trees (Binary Indexed Trees)

#	Category	Topic
31	Advanced	Dynamic Programming — 1D
32	Advanced	Dynamic Programming — 2D
33	Advanced	Greedy Algorithms
34	Advanced	Backtracking
35	Advanced	Bit Manipulation

Appendix B: Complexity Reference Table

Every topic page renders a complexity table analogous to the one below. This consolidated appendix gathers the complexity classes for the core linear and tree-based structures covered in the platform, matching the values driving each topic's "Complexity Dial" gauge.

Structure	Access	Search	Insertion	Deletion	Space
Array	O(1)	O(n)	O(n)	O(n)	O(n)
Singly Linked List	O(n)	O(n)	O(1)	O(1)	O(n)
Doubly Linked List	O(n)	O(n)	O(1)	O(1)	O(n)
Stack	O(n)	O(n)	O(1)	O(1)	O(n)
Queue	O(n)	O(n)	O(1)	O(1)	O(n)
Hash Map (avg.)	—	O(1)	O(1)	O(1)	O(n)
Binary Search Tree (avg.)	O(log n)	O(log n)	O(log n)	O(log n)	O(n)
AVL Tree (worst)	O(log n)	O(log n)	O(log n)	O(log n)	O(n)
Binary Heap	O(1) top	O(n)	O(log n)	O(log n)	O(n)
Trie (per word of length m)	O(m)	O(m)	O(m)	O(m)	O(alphabet x nodes)

B.1 Sorting Algorithm Complexity

Algorithm	Best	Average	Worst	Space	Stable?
Bubble Sort	O(n)	O(n^2)	O(n^2)	O(1)	Yes
Insertion Sort	O(n)	O(n^2)	O(n^2)	O(1)	Yes
Selection Sort	O(n^2)	O(n^2)	O(n^2)	O(1)	No
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes
Quick Sort	O(n log n)	O(n log n)	O(n^2)	O(log n)	No
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	No

Appendix C: Sample Auto-Graded Practice Problem

To make the grading pipeline concrete, this appendix walks through a single practice problem end-to-end: definition, hidden test suite, a learner's submission, and the grader's evaluation logic.

C.1 Problem Definition

Problem: "Valid Parentheses" — Given a string containing only the characters '(', ')', '{', '}', '[' and ']', determine whether the input string has properly matched and nested brackets. This problem is attached to the Stack topic, reinforcing the LIFO property taught on that page.

C.2 Reference / Learner Submission

```javascript
function isValid(s) {
  const stack = [];
  const pairs = { ')': '(', '}': '{', ']': '[' };
  for (const ch of s) {
    if (ch === '(' || ch === '{' || ch === '[') {
      stack.push(ch);
    } else {
      if (stack.pop() !== pairs[ch]) return false;
    }
  }
  return stack.length === 0;
}
```

C.3 Hidden Test Suite (server-side only)

Case	Input	Expected Output
1	"()"	true
2	"()[]{}"	true
3	"(]"	false
4	"([)]"	false
5	"{[]}"	true
6	"" (empty string)	true

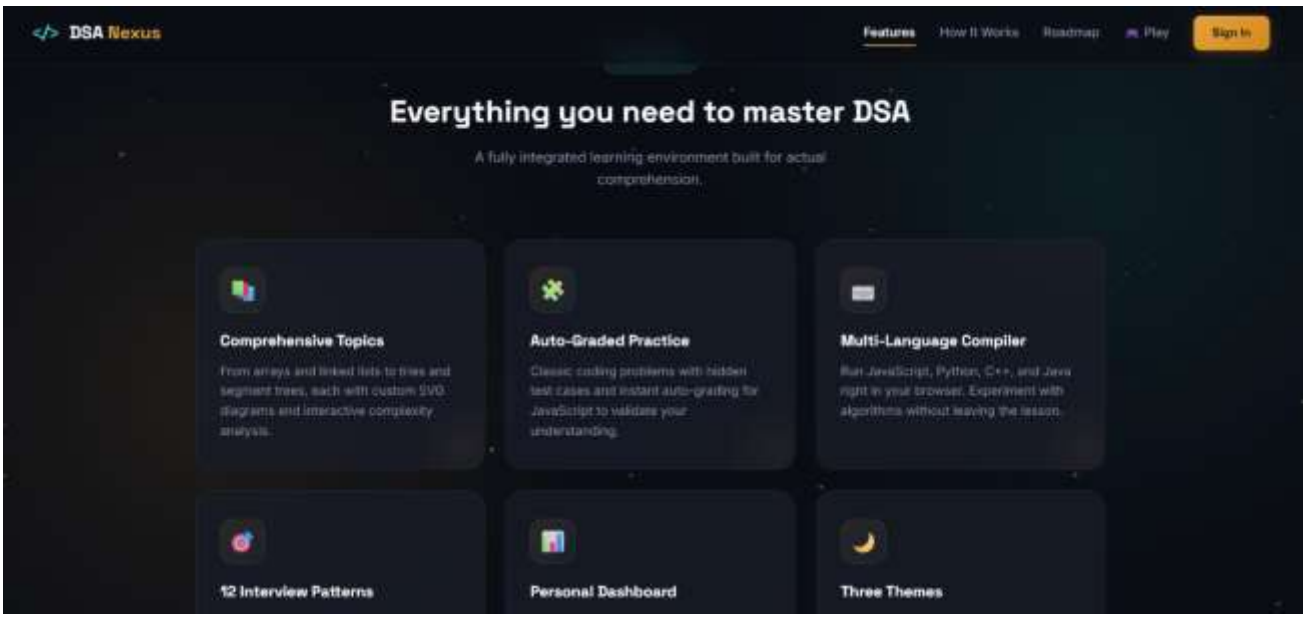
C.4 Grader Evaluation Logic

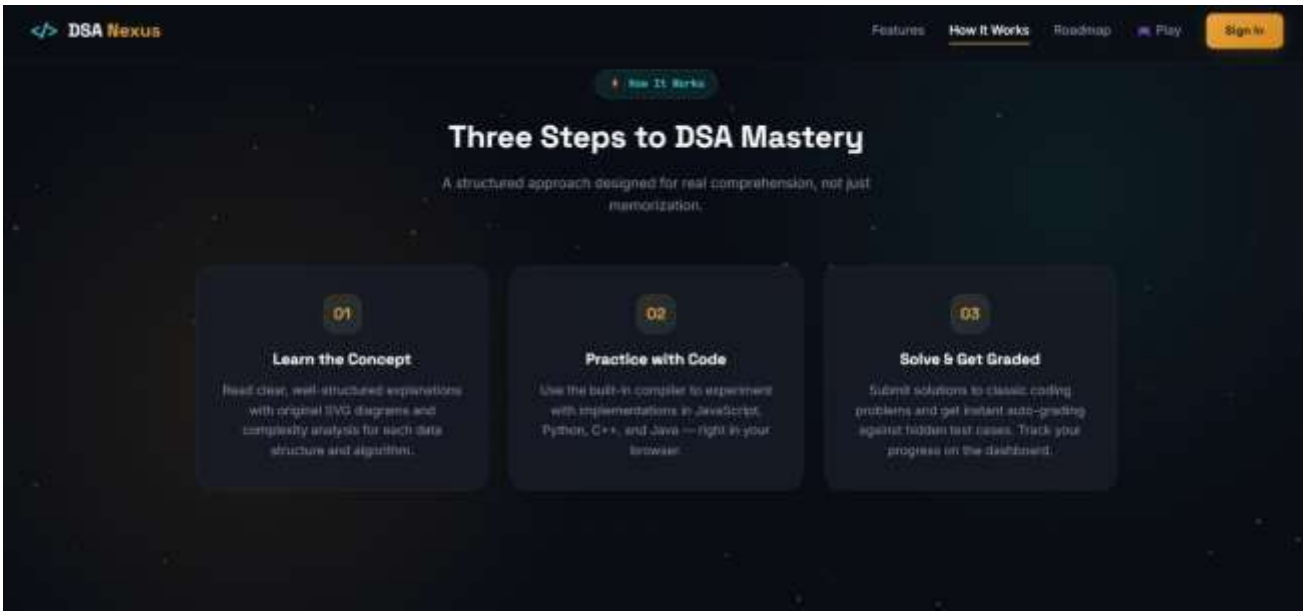
```javascript
// lib/grader.js  (excerpt — representative structure)
function grade(submissionFn, hiddenCases) {
  const results = hiddenCases.map((tc, i) => {
    let actual;
    try { actual = submissionFn(...tc.args); }
    catch (e) { return { case: i + 1, passed: false, error: e.message }; }
    return { case: i + 1, passed: actual === tc.expected };
  });
  const passedCount = results.filter(r => r.passed).length;
  return {
    verdict: passedCount === hiddenCases.length ? "Accepted" : "Wrong Answer",
    passedCount, total: hiddenCases.length, results,
  };
```

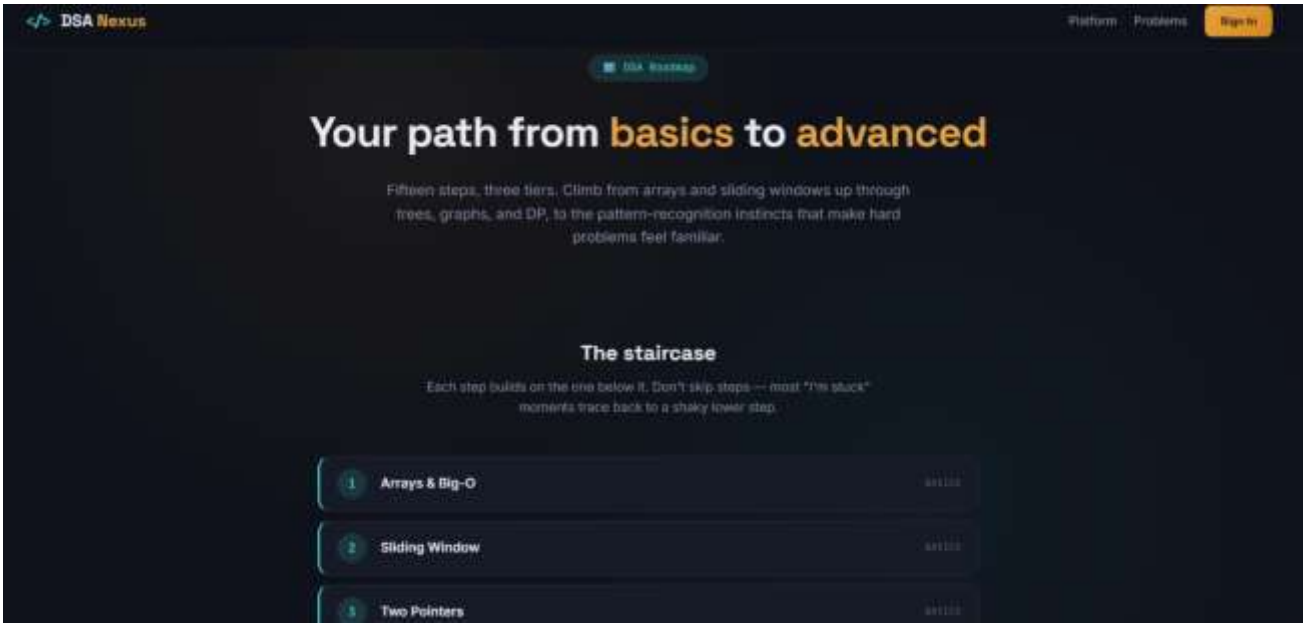
```
}
module.exports = { grade };
```

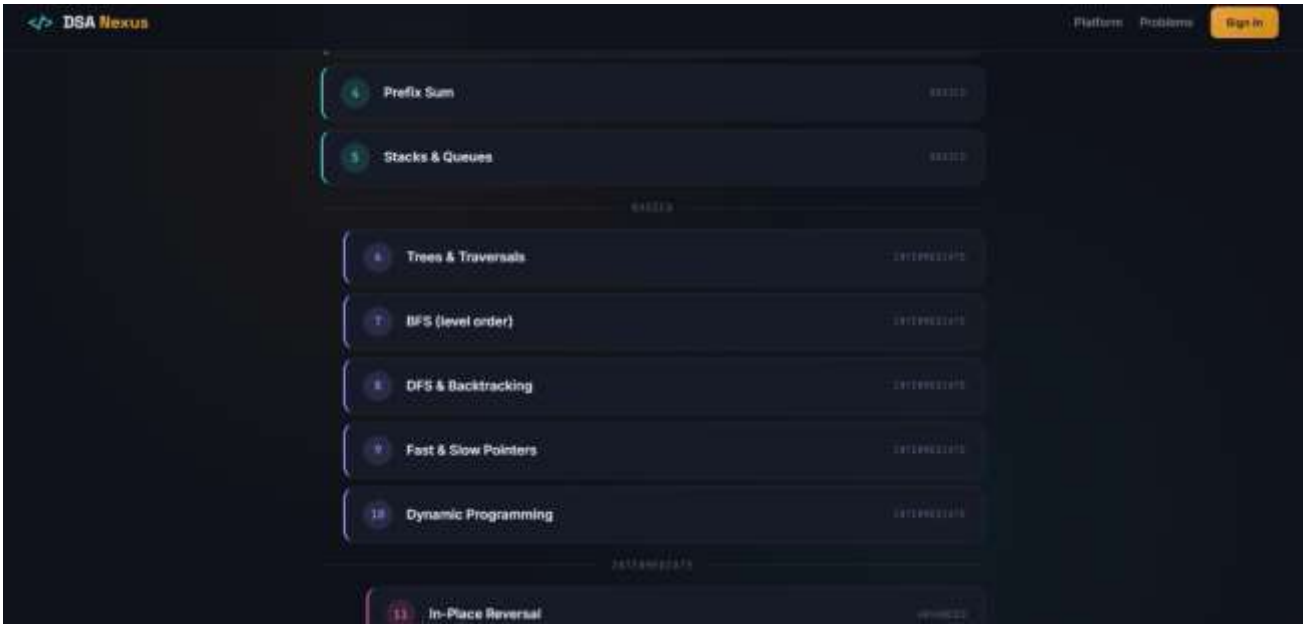
Note how the grader returns only the pass/fail status and case index — never the hidden input or expected value itself — matching production judge-platform behavior and preventing learners from reverse-engineering the hidden suite from the response payload.

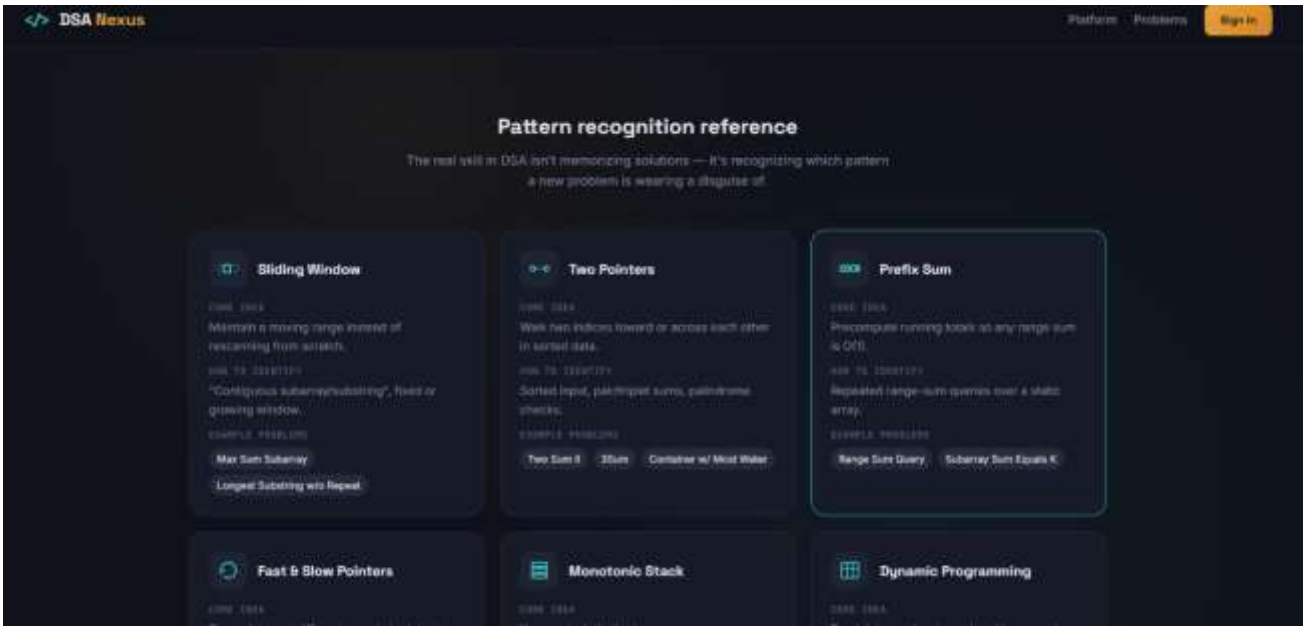
Landing Page:

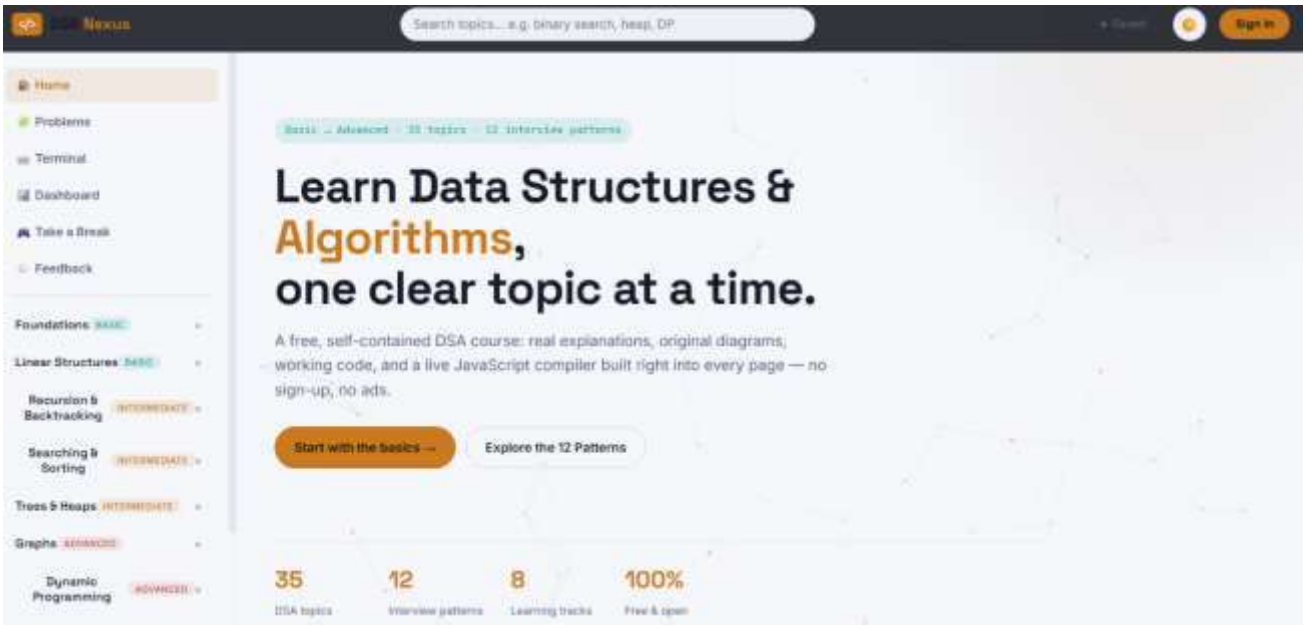



DSA Nexus
Features
How It Works
Roadmap
Play
Sign In
How It Works
Three Steps to DSA Mastery
A structured approach designed for real comprehension, not just memorization.
01
Learn the Concept
Read clear, well-structured explanations with original SVG diagrams and complexity analysis for each data structure and algorithm.
02
Practice with Code
Use the built-in compiler to experiment with implementations in JavaScript, Python, C++, and Java — right in your browser.
03
Solve & Get Graded
Submit solutions to classic coding problems and get instant auto-grading against hidden test cases. Track your progress on the dashboard.

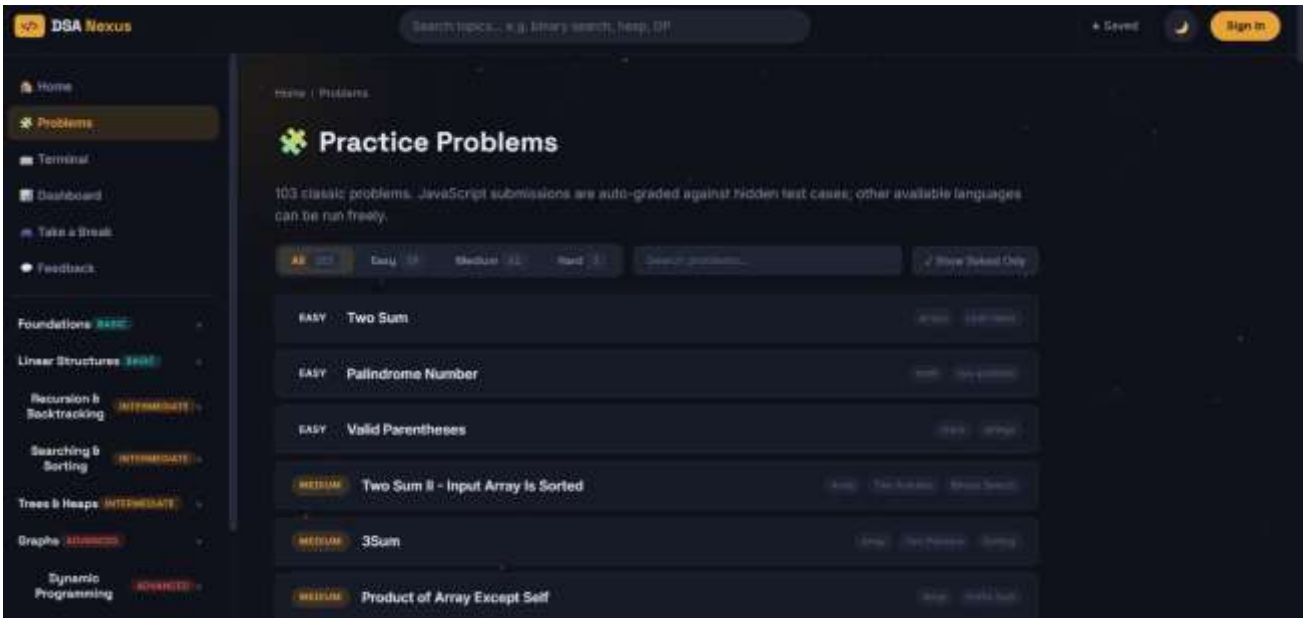

DSA Nexus
Platform
Problems
Sign In
DSA Roadmap
Your path from basics to advanced
Fifteen steps, three tiers. Climb from arrays and sliding windows up through trees, graphs, and DP, to the pattern-recognition instincts that make hard problems feel familiar.
The staircase
Each step builds on the one below it. Don't skip steps — most "I'm stuck" moments trace back to a shaky lower step.
1 Arrays & Big-O
2 Sliding Window
3 Two Pointers

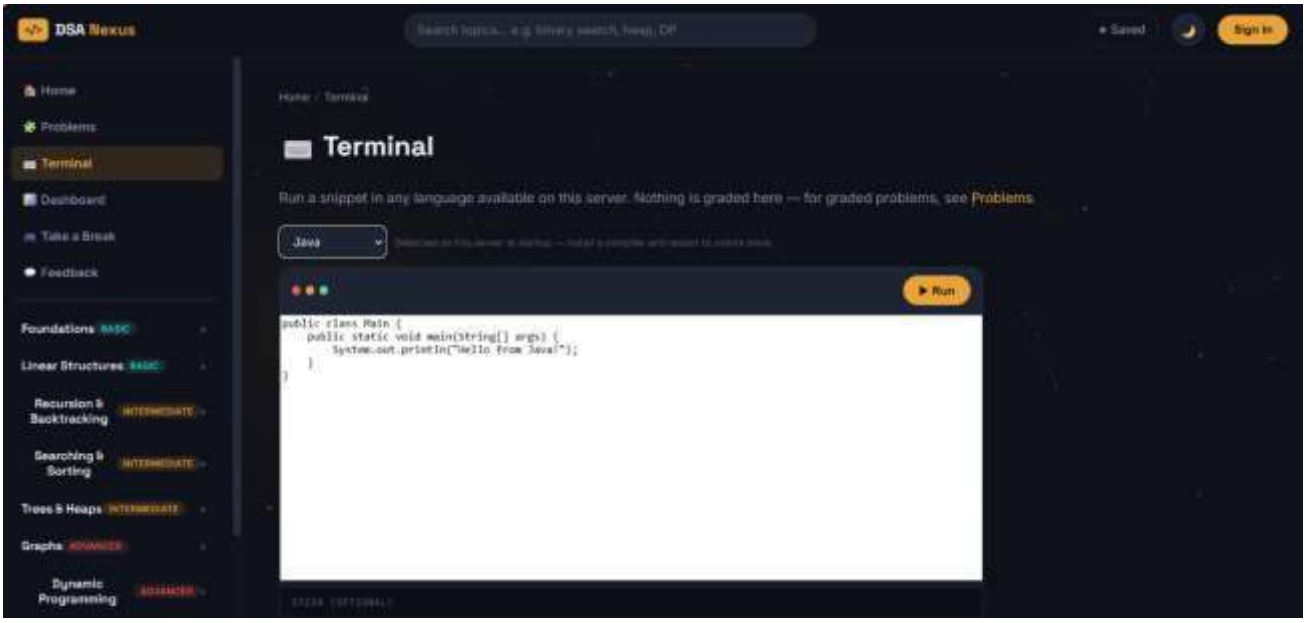

DSA Nexus
Platform
Problems
Sign In
4 Prefix Sum
5 Stacks & Queues
6 Trees & Traversals
7 BFS (level order)
8 DFS & Backtracking
9 Fast & Slow Pointers
10 Dynamic Programming
11 In-Place Reversal

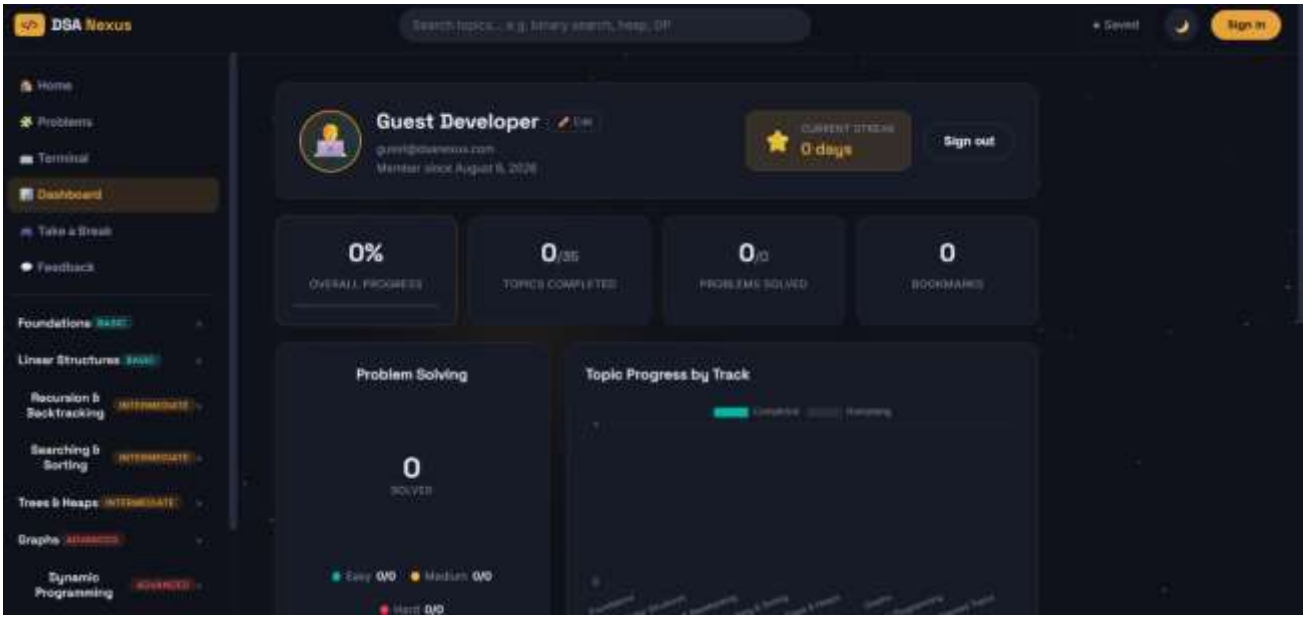

DSA Nexus
Pattern recognition reference
Sliding Window
Two Pointers
Prefix Sum
Fast & Slow Pointers
Monotonic Stack
Dynamic Programming

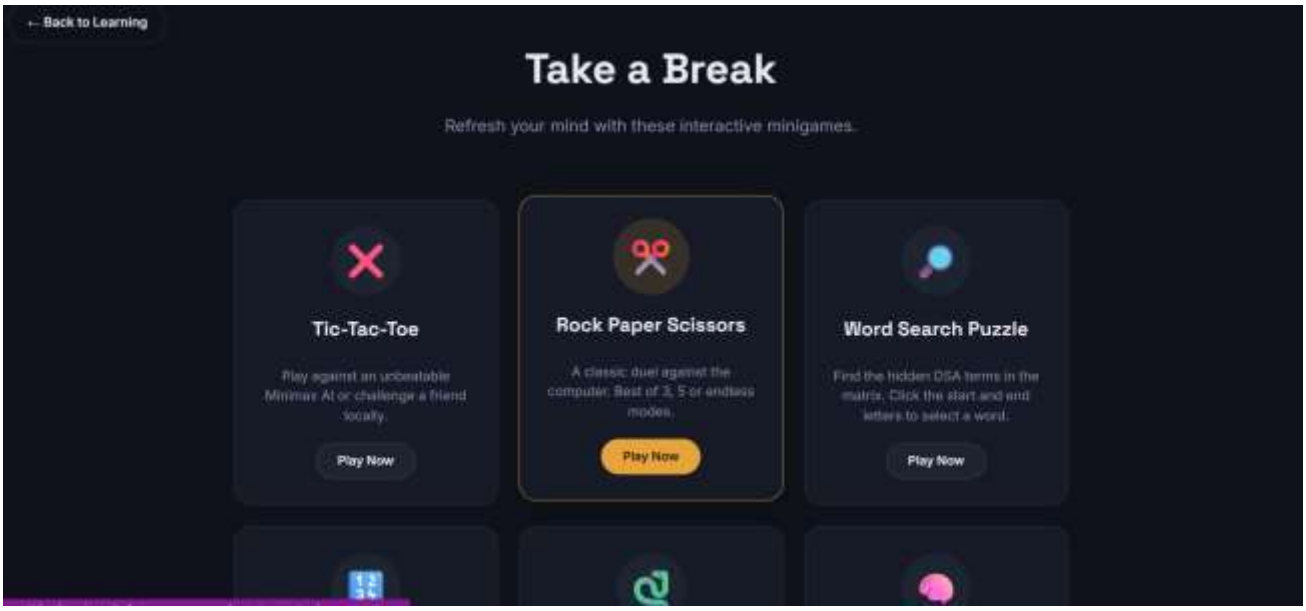

Learn Data Structures &
Algorithms,
one clear topic at a time.
A free, self-contained DSA course: real explanations, original diagrams, working code, and a live JavaScript compiler built right into every page — no sign-up, no ads.
Start with the basics →
Explore the 12 Patterns
35
12
8
100%


DSA Nexus
Learn Data Structures &
Algorithms,
one clear topic at a time.
A free, self-contained DSA course: real explanations, original diagrams, working code, and a live JavaScript compiler built right into every page — no sign-up, no ads.
Start with the basics →
Explore the 12 Patterns
35
12
8
100%

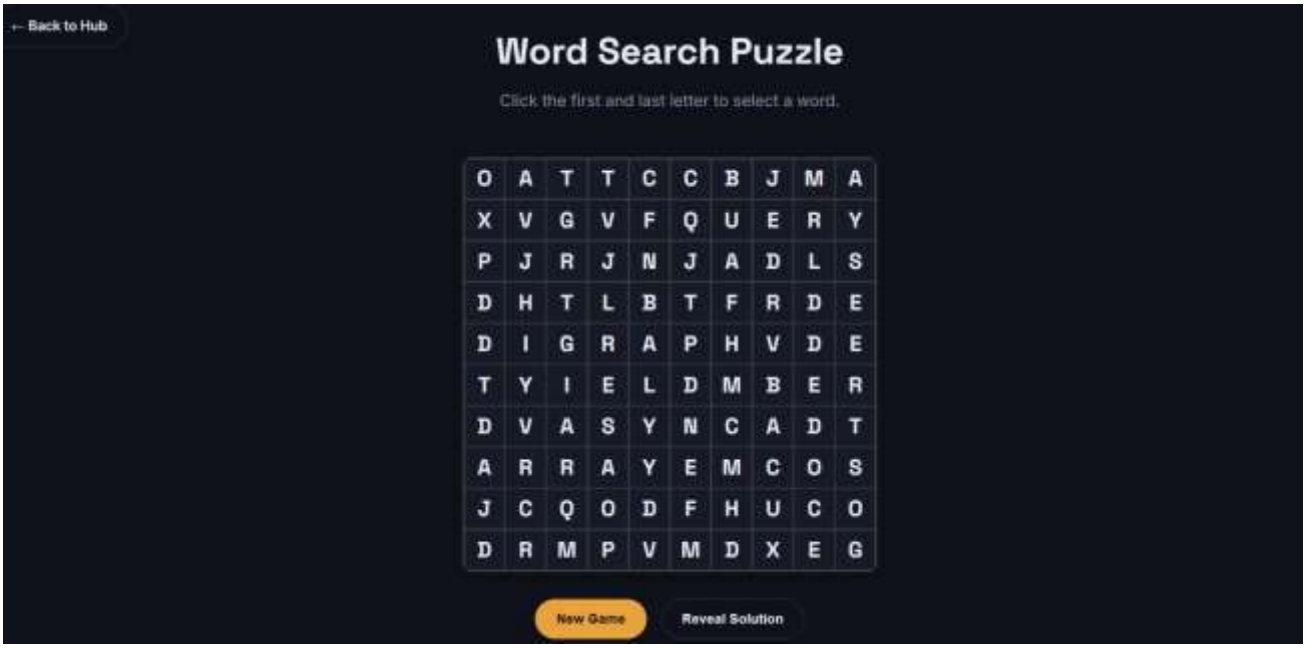

DSA Nexus
Practice Problems
103 classic problems. JavaScript submissions are auto-graded against hidden test cases; other available languages can be run freely.
EASY Two Sum
EASY Palindrome Number
EASY Valid Parentheses
MEDIUM Two Sum II - Input Array Is Sorted
MEDIUM 3Sum
MEDIUM Product of Array Except Self

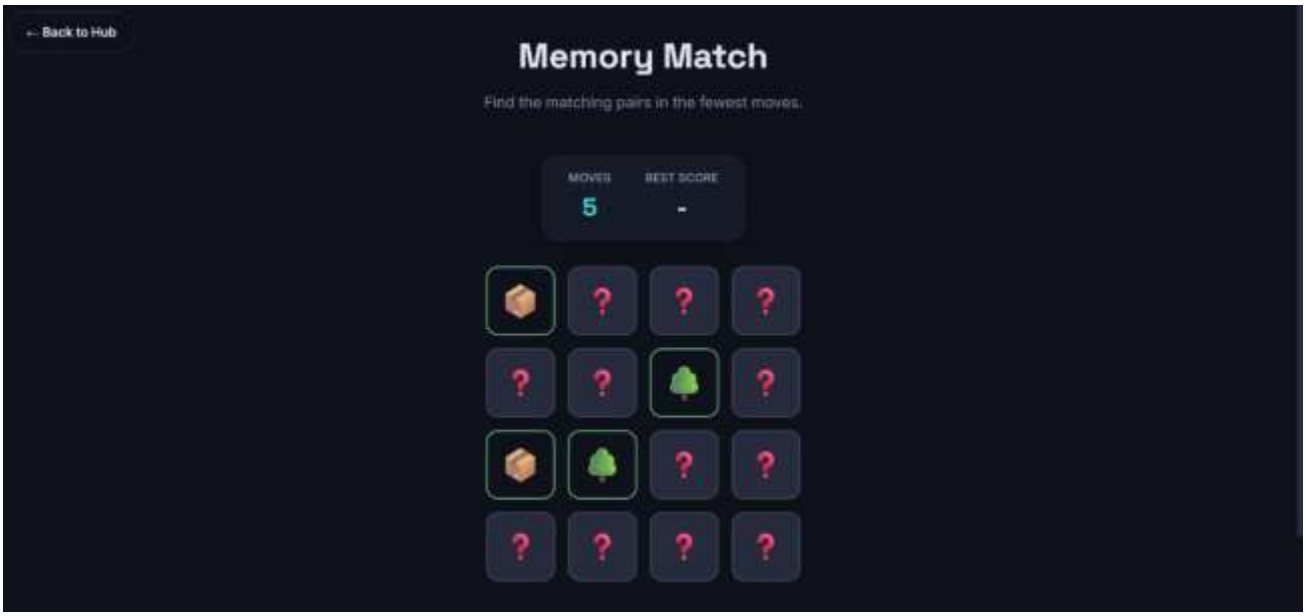

DSA Nexus
Terminal
Run a snippet in any language available on this server. Nothing is graded here — for graded problems, see Problems.
Java
Run
public class Main {
public static void main(String[] args) {
System.out.println("Hello from Java!");
}
}

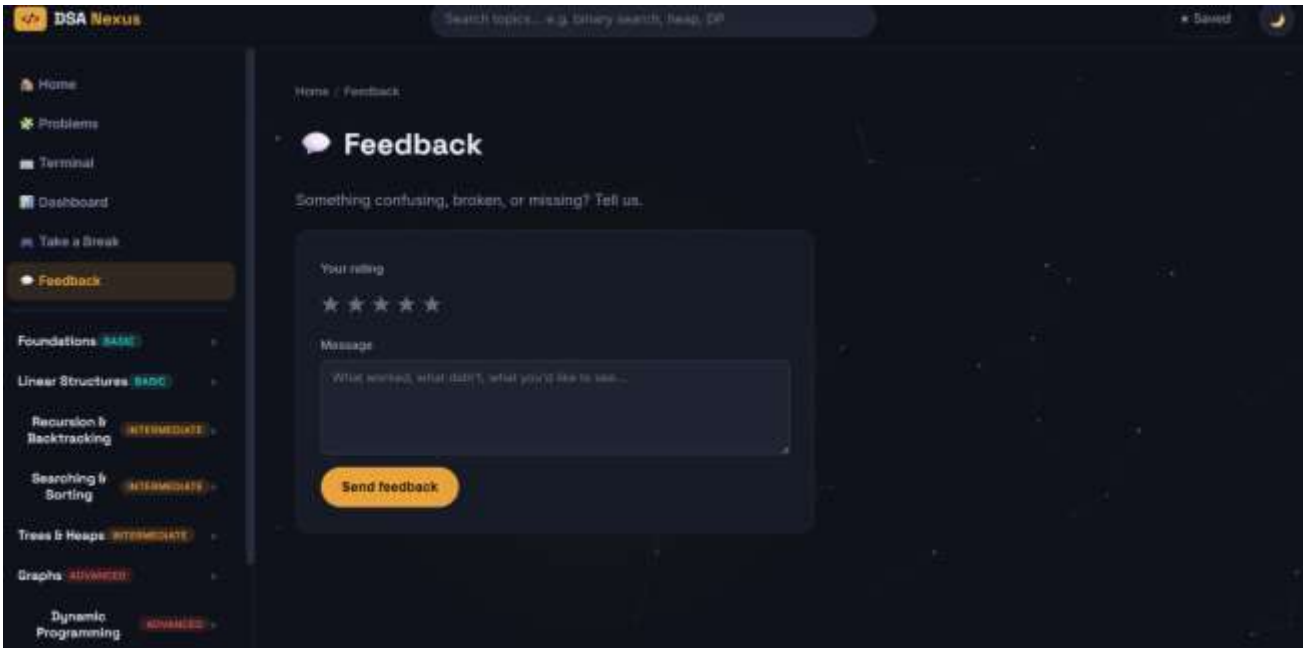

DSA Nexus
Guest Developer
Sign out
0%
Problem Solving
Topic Progress by Track

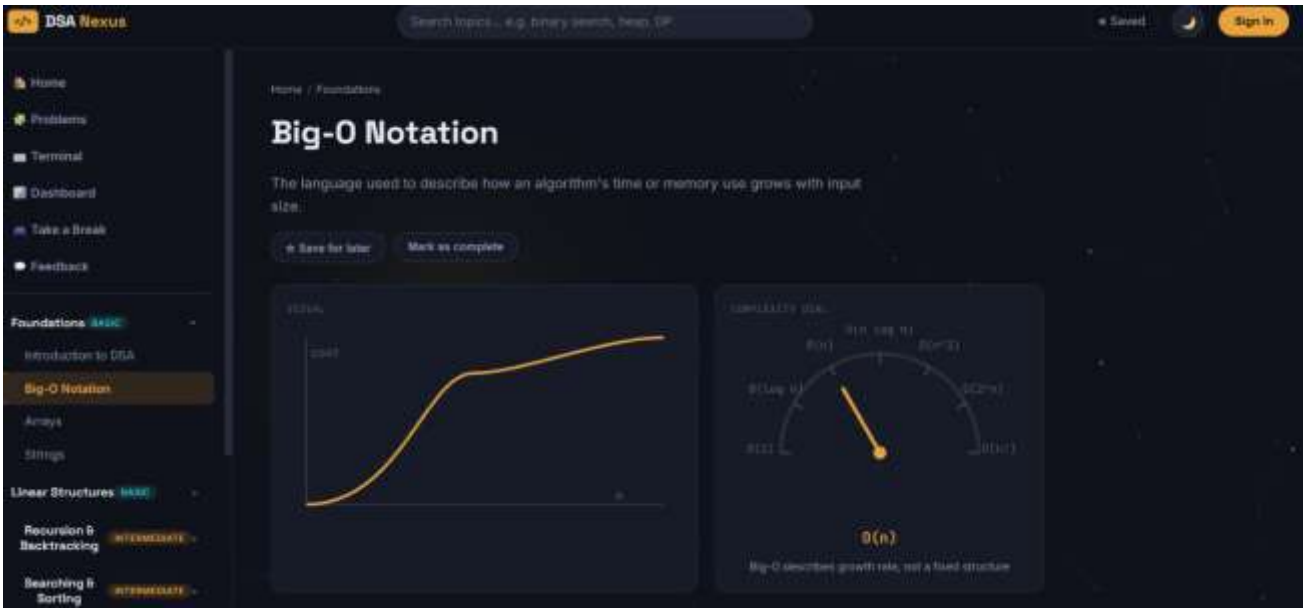

← Back to Learning
Take a Break
Refresh your mind with these interactive minigames.
Tic-Tac-Toe
Play against an unbeatable Minimax AI or challenge a friend locally.
Play Now
Rock Paper Scissors
A classic duel against the computer. Best of 3, 5 or endless modes.
Play Now
Word Search Puzzle
Find the hidden DSA terms in the matrix. Click the start and end letters to select a word.
Play Now

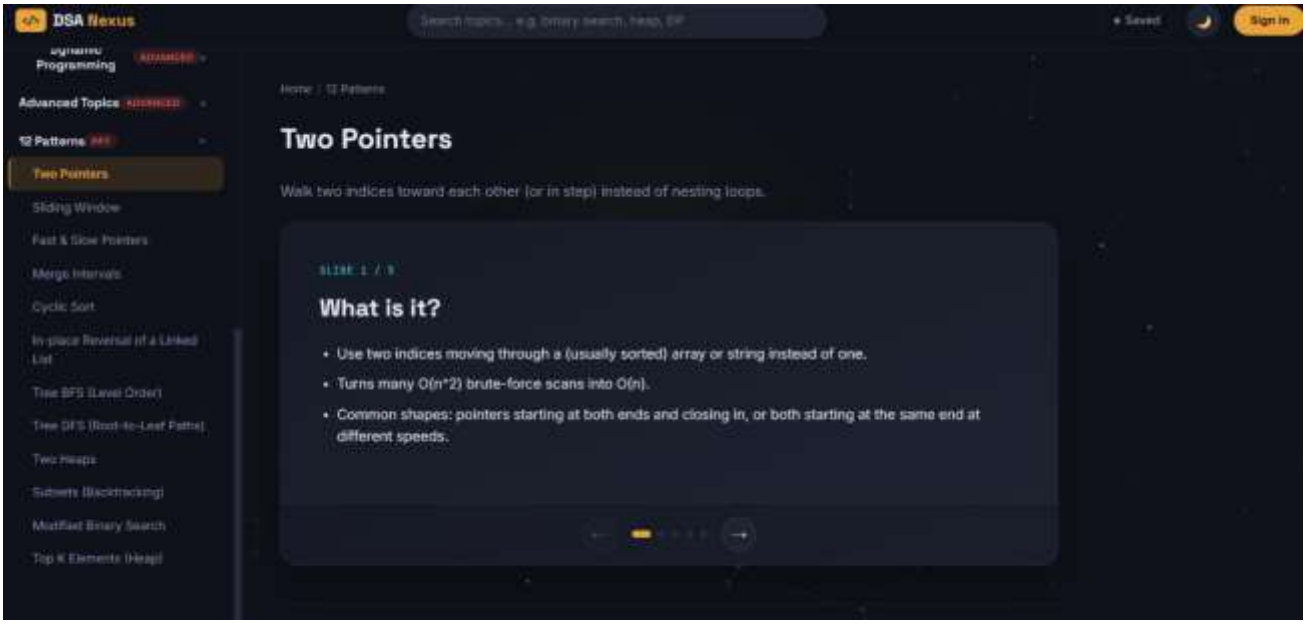

← Back to Learning
Tic-Tac-Toe
Play against the AI or a friend locally.
YOU (X) DRAWS AI (O)
0 0 0
X
O
Your turn!


← Back to Hub
Word Search Puzzle
Click the first and last letter to select a word.
O A T T C C B J M A
X V G V F Q U E R Y
P J R J N J A D L S
D H T L B T F R D E
D I G R A P H V D E
T Y I E L D M B E R
D V A S Y N C A D T
A R R A Y E M C O S
J C Q O D F H U C O
D R M P V M D X E G
New Game
Reveal Solution


← Back to Hub
Memory Match
Find the matching pairs in the fewest moves.
MOVES
5
BEST SCORE
-


DSA Nexus
Home
Problems
Terminal
Dashboard
Take a Break
Feedback
Foundations
Linear Structures
Recursion & Backtracking
Searching & Sorting
Trees & Heaps
Graphs
Dynamic Programming
Home / Feedback
Feedback
Something confusing, broken, or missing? Tell us.
Your rating
Message
Send feedback





Server.js file:

```
// server.js — DSA Nexus backend
// Serves: a marketing landing page ("/"), the learning app ("/app"),
// content APIs (topics/patterns/problems), auth (JWT + optional Google),
// a personal dashboard API, a multi-language code runner, and feedback.

require("dotenv").config({ quiet: true });
const express = require("express");
const compression = require("compression");
const helmet = require("helmet");
const cors = require("cors");
const rateLimit = require("express-rate-limit");
const path = require("path");

const { CATEGORIES, TOPICS, SCALE } = require("./data/topics");
const { PATTERNS } = require("./data/patterns");
const { runCode, AVAILABLE } = require("./lib/runner");

const authRoutes = require("./middleware/auth");
const meRoutes = require("./routes/me");
const problemsRoutes = require("./routes/problems");
const runRoutes = require("./routes/run");
```

```
const feedbackRoutes = require("./routes/feedback");

const app = express();
const PORT = process.env.PORT || 3000;

// ---------- Security & performance middleware ----------

app.use(helmet({
  contentSecurityPolicy: false,  // CSP breaks inline scripts; handle separately if needed
  crossOriginEmbedderPolicy: false
}));
app.use(cors());
app.use(compression());
app.use(express.json({ limit: "200kb" }));

// Rate-limit auth endpoints to prevent brute-force attacks
const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,   // 15 minutes
  max: 30,                    // 30 attempts per window
  standardHeaders: true,
  legacyHeaders: false,
  message: { error: "Too many attempts. Please try again in 15 minutes." }
});

// index:false — we control "/" and "/app" ourselves below (landing vs SPA)
app.use(express.static(path.join(__dirname, "public"), { index: false }));

// ---------- Content API (topics / patterns / search) ----------

app.get("/api/categories", (req, res) => {
  const withCounts = CATEGORIES.map(c => ({ ...c, count: TOPICS.filter(t => t.category === c.id).length }));
  res.json({ categories: withCounts, scale: SCALE });
});

app.get("/api/topics", (req, res) => {
  const list = TOPICS.map(({ id, category, title, summary, diagram, complexity }) => ({ id, category, title, summary, diagram, complexity
}));
  res.json({ total: list.length, topics: list });
});

app.get("/api/topics/:id", (req, res) => {
  const topic = TOPICS.find(t => t.id === req.params.id);
  if (!topic) return res.status(404).json({ error: "Topic not found" });
  res.json(topic);
});

app.get("/api/search", (req, res) => {
  const q = (req.query.q || "").toLowerCase().trim();
  if (!q) return res.json({ results: [] });
  const topicResults = TOPICS.filter(t => t.title.toLowerCase().includes(q) || t.summary.toLowerCase().includes(q))
    .map(({ id, title, category, summary }) => ({ id, title, category, summary, type: "topic" }));
  const patternResults = PATTERNS.filter(p => p.title.toLowerCase().includes(q) || p.tagline.toLowerCase().includes(q))
    .map(({ id, title, tagline }) => ({ id, title, category: "pattern", summary: tagline, type: "pattern" }));
  res.json({ results: [...topicResults, ...patternResults] });
});

app.get("/api/patterns", (req, res) => {
  const list = PATTERNS.map(({ id, title, tagline, diagram, slides }) => ({ id, title, tagline, diagram, slideCount: slides.length }));
  res.json({ total: list.length, patterns: list });
});

app.get("/api/patterns/:id", (req, res) => {
  const pattern = PATTERNS.find(p => p.id === req.params.id);
  if (!pattern) return res.status(404).json({ error: "Pattern not found" });
  res.json(pattern);
});
```

```
// ---------- Quick JS "try it yourself" compiler (used on topic pages) ----------

app.post("/api/compile", async (req, res) => {
  const result = await runCode({ language: "javascript", code: (req.body || {}).code });
  res.json({ ok: result.ok, output: result.stdout, error: result.ok ? undefined : result.stderr, ms: result.ms });
});

// ---------- Auth / dashboard / problems / terminal / feedback ----------

app.use("/api/auth", authLimiter, authRoutes);
app.use("/api/me", meRoutes);
app.use("/api/problems", problemsRoutes);
app.use("/api/run", runRoutes);
app.use("/api/feedback", feedbackRoutes);
app.get("/api/languages", (req, res) => res.json({ available: AVAILABLE }));

app.get("/health", (req, res) => res.json({ status: "ok" }));

// ---------- Page routing: landing page at "/", the SPA at "/app" ----------

app.get("/", (req, res) => res.sendFile(path.join(__dirname, "public", "welcome.html")));
app.get("/roadmap", (req, res) => res.sendFile(path.join(__dirname, "public", "roadmap.html")));
app.get(["/app", "/app/*"], (req, res) => res.sendFile(path.join(__dirname, "public", "index.html")));
app.get(["/login", "/signin", "/register", "/signup"], (req, res) => res.sendFile(path.join(__dirname, "public", "login.html")));
app.get("/games", (req, res) => res.sendFile(path.join(__dirname, "public", "games-hub.html")));
app.get(["/games/tic-tac-toe", "/play/tic-tac-toe"], (req, res) => res.sendFile(path.join(__dirname, "public", "tic-tac-toe.html")));
app.get("/games/rock-paper-scissors", (req, res) => res.sendFile(path.join(__dirname, "public", "rock-paper-scissors.html")));
app.get("/games/word-game", (req, res) => res.sendFile(path.join(__dirname, "public", "word-game.html")));
app.get("/games/sudoku", (req, res) => res.sendFile(path.join(__dirname, "public", "sudoku.html")));
app.get("/games/snake-and-ladder", (req, res) => res.sendFile(path.join(__dirname, "public", "snake-and-ladder.html")));
app.get("/games/memory-match", (req, res) => res.sendFile(path.join(__dirname, "public", "memory-match.html")));

app.get("*", (req, res) => res.redirect("/"));

if (require.main === module) {
  app.listen(PORT, () => {
    console.log(`DSA Nexus running at http://localhost:${PORT}`);
    console.log(`Languages available for the terminal: ${Object.entries(AVAILABLE).filter(([, v]) => v).map(([k]) => k).join(", ")}`);
    if (!process.env.GOOGLE_CLIENT_ID) console.log(`Google Sign-In: disabled (set GOOGLE_CLIENT_ID in .env to enable)`);
  });
}

module.exports = app;
```